

COMP2271: Computer Graphics

Part 4: Geometry & Texture Mapping

Dr. Frederick Li

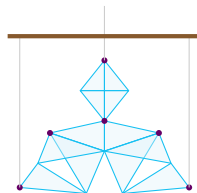
Department of Computer Science
Durham University

The Invisible Skeleton

From Mathematical Points to Physical Form

When you look at a digital character, you see skin, cloth, and texture. But beneath the surface lies a **mathematical primal**.

- Every 3D object is a **wireframe ghost**, i.e., a collection of points in space.



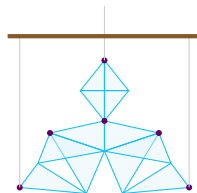
GEOMETRIC PRIMITIVE MESH

The Invisible Skeleton

From Mathematical Points to Physical Form

When you look at a digital character, you see skin, cloth, and texture. But beneath the surface lies a **mathematical primal**.

- Every 3D object is a **wireframe ghost**, i.e., a collection of points in space.
- It is a hollow shell constructed from thousands of rigid, connecting **triangles**.



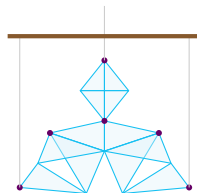
GEOMETRIC PRIMITIVE MESH

The Invisible Skeleton

From Mathematical Points to Physical Form

When you look at a digital character, you see skin, cloth, and texture. But beneath the surface lies a **mathematical primal**.

- Every 3D object is a **wireframe ghost**, i.e., a collection of points in space.
- It is a hollow shell constructed from thousands of rigid, connecting **triangles**.
- Like a **marionette**, it has no weight or mass. It only exists as a set of instructions.



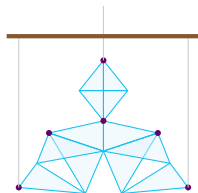
GEOMETRIC PRIMITIVE MESH

The Invisible Skeleton

From Mathematical Points to Physical Form

When you look at a digital character, you see skin, cloth, and texture. But beneath the surface lies a **mathematical primal**.

- Every 3D object is a **wireframe ghost**, i.e., a collection of points in space.
- It is a hollow shell constructed from thousands of rigid, connecting **triangles**.
- Like a **marionette**, it has no weight or mass. It only exists as a set of instructions.



GEOMETRIC PRIMITIVE MESH

The Programmer's Reality

You are the architect of this ghost. You must explicitly define every **Vertex** (3D coordinates) and every **Index** (the connectivity logic) to breathe life into the void.

1 Geometry & Meshes

2 Texture Mapping

3 Conclusion

The Blueprint of Virtual Worlds

- Before a scene can be rendered, we must first define the **shape** of its objects.
- This is the foundational layer of 3D graphics, i.e., the geometric description of everything we see.

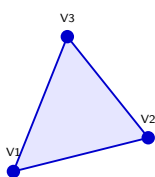
- Before a scene can be rendered, we must first define the **shape** of its objects.
- This is the foundational layer of 3D graphics, i.e., the geometric description of everything we see.
- Every complex 3D model, from a game character to a skyscraper, is constructed from a collection of simple geometric shapes called **primitives**.

The Blueprint of Virtual Worlds

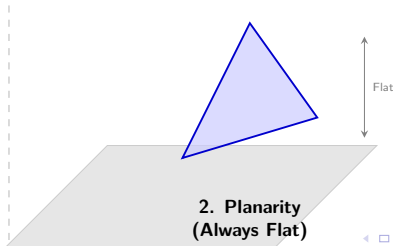
- Before a scene can be rendered, we must first define the **shape** of its objects.
- This is the foundational layer of 3D graphics, i.e., the geometric description of everything we see.
- Every complex 3D model, from a game character to a skyscraper, is constructed from a collection of simple geometric shapes called **primitives**.
- In modern computer graphics, the most fundamental primitive is the **triangle**.

Why Triangles?

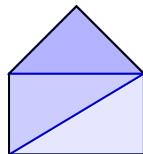
- **Simplicity:** Defined by just three vertices.
- **Planarity:** Three vertices are guaranteed to lie on the same plane, which simplifies rendering calculations immensely.
- **Versatility:** Any complex polygon can be decomposed into a set of triangles.



1. Simplicity
(Just 3 Points)



2. Planarity
(Always Flat)



3. Versatility
(Decomposition)

What is a Mesh?

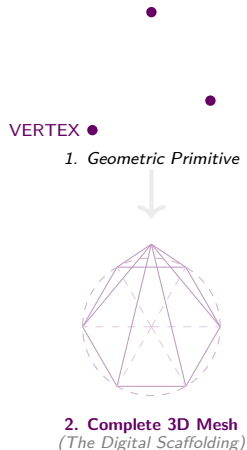
The Digital Scaffolding of Form

The Concept

A **Mesh** is the collection of vertices, edges, and faces that define a polyhedral object. It acts as the **digital scaffolding** that creates the surface.

The Primary Data Lists:

- **Vertices:** The unique points (corners) in 3D space.



What is a Mesh?

The Digital Scaffolding of Form

The Concept

A **Mesh** is the collection of vertices, edges, and faces that define a polyhedral object. It acts as the **digital scaffolding** that creates the surface.

The Primary Data Lists:

- **Vertices:** The unique points (corners) in 3D space.
- **Indices:** The "connect-the-dots" recipe that forms faces.



1. Geometric Primitive



2. Complete 3D Mesh
(The Digital Scaffolding)

What is a Mesh?

The Digital Scaffolding of Form

The Concept

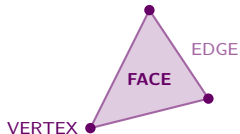
A **Mesh** is the collection of vertices, edges, and faces that define a polyhedral object. It acts as the **digital scaffolding** that creates the surface.

The Primary Data Lists:

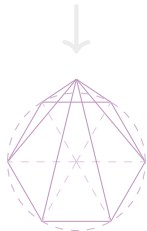
- **Vertices:** The unique points (corners) in 3D space.
- **Indices:** The "connect-the-dots" recipe that forms faces.

The Build-up

Points (Vertices) → Lines (Edges) → Surfaces (Faces)
→ **Complete Mesh**



1. Geometric Primitive



2. Complete 3D Mesh
(The Digital Scaffolding)

A vertex is much more than just a 3D position. It is a container for various **attributes** that describe the surface at that specific point.

Common Attributes:

- **Position:** The mandatory (x, y, z) coordinates.
- **Normal Vector:** A vector pointing outwards, perpendicular to the surface. Crucial for lighting.
- **Texture Coordinates (UVs):** A 2D coordinate that maps the vertex to a point on a 2D image (texture).
- **Colour:** A per-vertex colour.

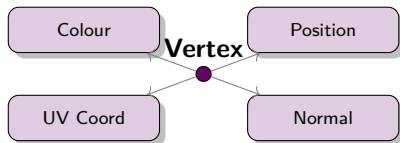


Figure: A single vertex can hold multiple pieces of data.

Vertex Buffer Object (VBO)

This collection of vertex data is organised and sent to the GPU, where it is stored in a dedicated memory buffer called a **Vertex Buffer Object (VBO)**.

The Connectivity Recipe

Mapping Vertices to Faces

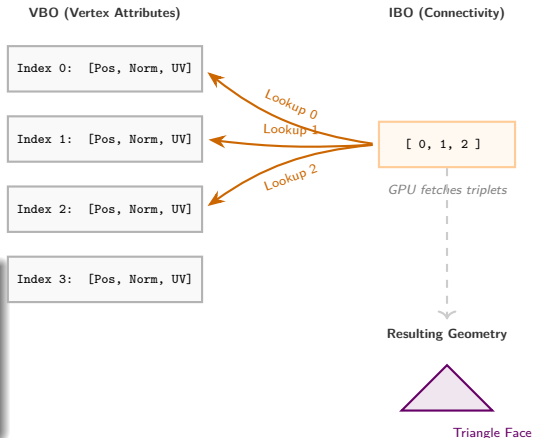
If the VBO is the "material," the **Index List** is the "blueprint".

- **Connect-the-Dots:** A list of integers pointing to specific positions in the vertex list.
- **Triangulation:** The GPU fetches these in **triplets** to define each face.

The GPU's Instruction

For an index list [0, 1, 2]:

- 1 Fetch vertex at position 0 (V_0).
- 2 Fetch vertex at position 1 (V_1).
- 3 Fetch vertex at position 2 (V_2).
- 4 Rasterise the triangle (V_0, V_1, V_2).



Index Buffer Object (IBO)

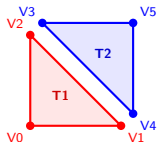
This "recipe" is stored in its own high-speed memory on the GPU, known as the **IBO** or Element Array Buffer.

Optimisation: Eliminating Data Redundancy

Why We Use Index Buffers (IBOs)

The Naive Way

(Independent Triangles)

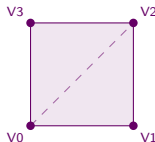


Independent Triangles (6 Total Vertices)

- Each triangle requires 3 unique vertices.
- Total: **6 Vertices** for one square.
- Shared corners are duplicated in memory.
- **Result: Redundant VRAM usage.**

The Optimised Way

(Shared Vertex Pool)

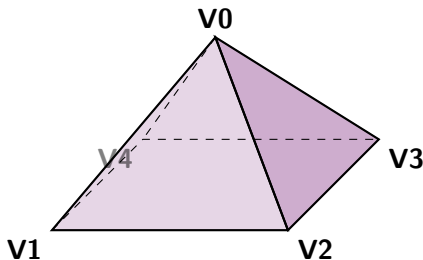


Shared Vertex Pool (4 Points)

- Define only the **4 unique corners**.
- Index list: [0,1,2] and [0,2,3].
- Total: **4 Vertices + 6 Indices**.
- **Result: Significant memory savings.**

The Scaling Impact

For a complex mesh, vertices are typically shared by **3 to 6 triangles**. Using an IBO reduces vertex data size by roughly **60–80%**, drastically improving performance.



Vertex Buffer (VBO)

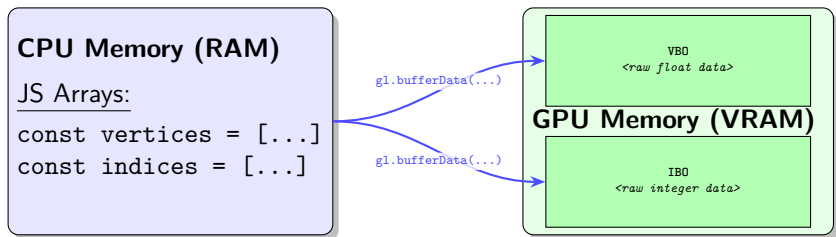
0: V0 (x,y,z...)
1: V1 (x,y,z...)
2: V2 (x,y,z...)
3: V3 (x,y,z...)
4: V4 (x,y,z...)

Index Buffer (IBO)

% Front Face
[0, 1, 2]
% Right Face
[0, 2, 3]
% Back Face
[0, 3, 4]
% etc...

Vertex **V0** is shared by four triangles. Instead of storing its data four times, we store it once and reference it four times in the index buffer.

Geometric data is defined in our application (CPU memory) and must be transferred to the GPU's dedicated memory (VRAM) to be rendered.



WebGL Buffer Objects

WebGL provides **VBOs** and **IBOs** as mechanisms to allocate and manage chunks of memory on the GPU for storing this data.

The Three-Step Process:

- 1 **Create:** 'gl.createBuffer()' reserves a buffer handle on the GPU.
- 2 **Bind:** 'gl.bindBuffer(gl.ARRAY_BUFFER, ...)' activates the buffer, making it the current target for data operations.
- 3 **Buffer Data:** 'gl.bufferData(...)' copies the data from your JavaScript array (CPU) into the currently bound buffer (GPU).

```
// 1. Create a buffer object on the GPU
const vertexBuffer = gl.createBuffer();
// 2. Bind the buffer as the current ARRAY_BUFFER
gl.bindBuffer(gl.ARRAY_BUFFER, vertexBuffer);
// 3. Copy the JavaScript vertex data into the bound buffer
// Note: Data is converted to a Float32Array for direct GPU compatibility.
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array(vertices), gl.STATIC_DRAW);
```

gl.STATIC_DRAW

This is a hint to WebGL about how we will use the data. `STATIC_DRAW` means we intend to load the data once and use it for drawing many times, allowing for internal optimizations.

Vertex Attribute Pointers

Decoding the Raw Memory Stream

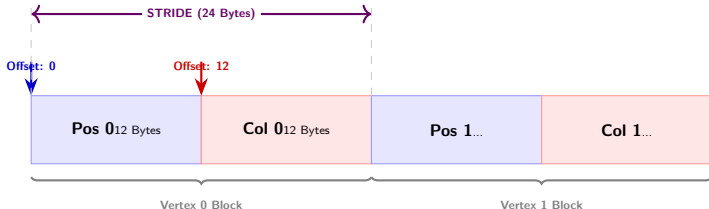
The VBO is just a **raw block of bytes** on the GPU. The shader needs a "blueprint" to interpret this stream. We use `gl.vertexAttribPointer` to define this layout.

1. Data Format (The "What")

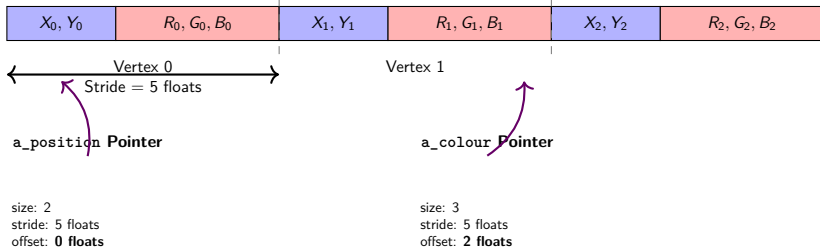
- **Size:** Number of components (e.g., 3 for a vec3 position).
- **Type:** Data format (e.g., `gl.FLOAT = 4` bytes).

2. Navigation (The "Where")

- **Stride:** Bytes to skip to reach the start of the **next** vertex.
- **Offset:** Bytes from the start to find **this** attribute.



Interleaved VBO: [Pos (2 floats), Colour (3 floats)]



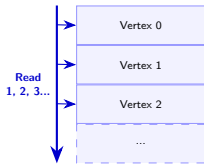
The **stride** is the size of a whole vertex block, while the **offset** specifies where each attribute starts within that block.

Triggering the Pipeline

Sequential vs. Indexed Rendering

After setup (Shaders, Buffers, Pointers), we issue a single command to wake up the GPU.

1. Sequential Mode (No Index Buffer)

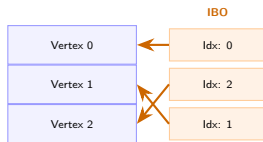


```
gl.drawArrays(...)
```

- **Logic:** Reads VBO from top to bottom.
- **Use Case:** Simple lines or points where data isn't shared.

```
gl.drawArrays(mode, first, count)
```

2. Indexed Mode (Using IBO Recipe)



```
gl.drawElements(...)
```

- **Logic:** IBO dictates the order. Allows jumping/reuse.
- **Use Case:** Complex 3D meshes (Optimization).

```
gl.drawElements(mode, count, ...)
```

This command launches the Vertex Shader → Rasterizer → Fragment Shader pipeline.

Why Learn VBOs & IBOs?

Going Behind the Abstraction Curtain

Three.js primitives ('BoxGeometry', etc.) are great, but they are just wrappers around the real GPU machinery.

1. The Abstraction (Three.js Helpers)

```
new BoxGeometry()
```

"Just give me a cube."
Easy, but inflexible.

Limitations

- Limited to standard shapes.
- Inefficient for drawing 10,000 moving objects (High Draw Call overhead).
- Hard to debug when shaders break.

2. The Reality (VBO & IBO) (Raw GPU Data)

VBO (Vertex Buffer)

[x, y, z, u, v, nx, ny...]

IBO (Index Buffer)

[0, 1, 2, 2, 3, 0...]

Why you need this:

- **Custom Geometry:** Procedural terrain (Minecraft-style), soft bodies, or cloth.
- **Performance:** Merging 1,000 objects into one buffer (Batching) to save CPU cycles.
- **Custom Attributes:** Passing unique data (e.g., "Health", "Temperature") directly to vertices.

Modern libraries like **Three.js** handle the low-level WebGL details for us, allowing us to focus on building the scene.

The process becomes much simpler:

- ➊ **Define Geometry:** The shape (vertices, indices, etc.). Three.js has built-in primitives like planes, boxes, and spheres.
- ➋ **Define Material:** The appearance (colour, texture, reaction to light).
- ➌ **Create Mesh:** Combine the geometry and material into a single object.
- ➍ **Add to Scene:** Place the mesh in the virtual world to be rendered.

```
// 1. Geometry (a 1x1 plane)
const geometry = new THREE.PlaneGeometry(1, 1);
// 2. Material (a simple green colour)
const material = new THREE.MeshBasicMaterial({ color: 0x00ff00 });
// 3. Mesh (Shape + Appearance)
const squareMesh = new THREE.Mesh(geometry, material);

// 4. Add to the scene
scene.add(squareMesh);
```

Behind the scenes, Three.js is creating VBOs, IBOs, setting attribute pointers, and making the 'gl.drawElements' call for you.

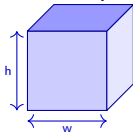
Built-in Geometries in three.js

Primitives for Quick Scene Construction

Three.js provides helper functions to generate mesh data (vertices & faces) for common shapes automatically.

1. Box (Cube)

```
new THREE.BoxGeometry(w, h, d);
```



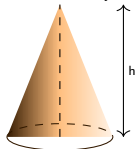
2. Sphere

```
new THREE.SphereGeometry(r, seg);
```



3. Cone

```
new THREE.ConeGeometry(r, h, seg);
```



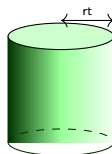
4. Plane (2D)

```
new THREE.PlaneGeometry(w, h);
```



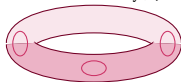
5. Cylinder

```
new THREE.CylinderGeometry(rt, rb, h);
```



6. Torus (Donut)

```
new THREE.TorusGeometry(r, tube);
```



Texture Mapping

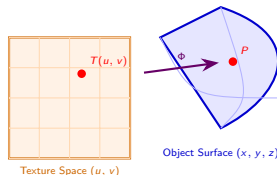
Decoupling Detail from Structure

In Computer Graphics, we separate the **macro-structure** (Shape) from the **micro-structure** (Appearance).

The Definition

Texture mapping is a function Φ that maps a point from a 2D Texture Space (u, v) to a 3D Object Space (x, y, z) .

$$(u, v) \xrightarrow{\Phi} (x, y, z) \xrightarrow{\text{Shader}} \text{Colour}$$



Why is this vital?

- **Geometric Economy:** Modeling every brick in a wall as polygons would require millions of triangles (High Cost).
- **Visual Density:** A single quad (2 triangles) can hold a 4K texture with millions of distinct color details (Low Cost).

The Trade-off

Geometry = Expensive ($O(N)$).
Textures = Cheap ($O(1)$ lookup).

Forward vs. Backward Mapping

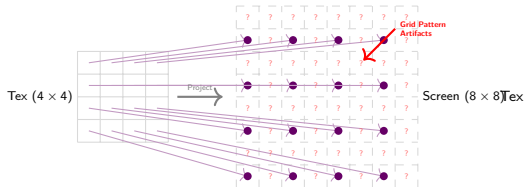
How do we connect pixels to texels?

1. Forward Mapping (Source $\rightarrow$ Screen)

Iterates over the **Texture Image**.

Projects each texel onto the screen.

Problem: Holes appear if texture is enlarged.

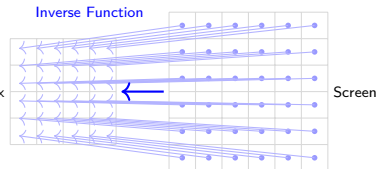


2. Backward Mapping (Screen $\rightarrow$ Source)

Iterates over the **Screen Pixels**.

Finds which texel covers the pixel center.

Benefit: Guaranteed coverage (No holes).



The Industry Standard

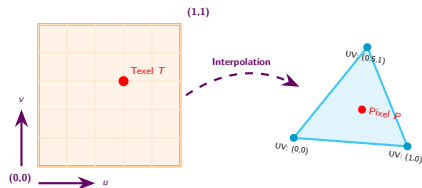
Modern GPUs (WebGL/OpenGL) rely on **Backward Mapping** during the Rasterization stage to ensure every pixel on the screen has a valid color.

Defining UV Coordinates

Mapping 2D Logic to 3D Form

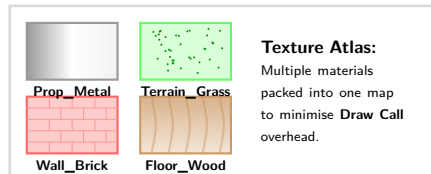
What are u and v ?

- Since (x, y, z) are already used for 3D space, we use u and v to describe the 2D axes of a texture.
- u (**Horizontal**): Ranges from 0.0 (left) to 1.0 (right).
- v (**Vertical**): Ranges from 0.0 (bottom) to 1.0 (top).



The Mapping Responsibility:

- Every **Vertex** stores a (u, v) pair as an attribute.
- The **Rasteriser** interpolates these values across the triangle's face.
- The **Fragment Shader** uses the result to "sample" the texture.



Practice: Texture Atlas

A **Texture Atlas** packs multiple sub-textures into one large image. This allows the GPU to render different surfaces (wood, brick, etc.) in a **single draw call** by shifting the UV coordinate offsets for different vertices.

Figure: Top: UV mapping logic. Bottom: Visual representation of a Texture Atlas.

The Origami Paradox

Rethinking How We Paint 3D Objects

We have spent this lecture building solid 3D shapes from triangles. It feels like we are sculptors, carving objects out of digital marble.

The Origami Paradox

Rethinking How We Paint 3D Objects

We have spent this lecture building solid 3D shapes from triangles. It feels like we are sculptors, carving objects out of digital marble.

But here is the paradox: To paint these 3D objects, we have to think in 2D again.

The Origami Paradox

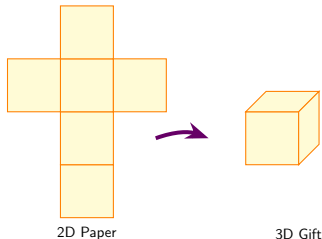
Rethinking How We Paint 3D Objects

We have spent this lecture building solid 3D shapes from triangles. It feels like we are sculptors, carving objects out of digital marble.

But here is the paradox: To paint these 3D objects, we have to think in 2D again.

Imagine you want to wrap a Christmas present. The box is 3D. The wrapping paper is 2D.

Texture Mapping is exactly this process in reverse. We don't paint on the 3D object; we take the 3D object, slice it open, and flatten it onto a 2D image like a piece of origami paper.



The Origami Paradox

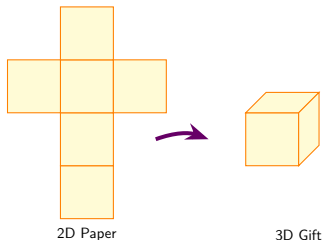
Rethinking How We Paint 3D Objects

We have spent this lecture building solid 3D shapes from triangles. It feels like we are sculptors, carving objects out of digital marble.

But here is the paradox: To paint these 3D objects, we have to think in 2D again.

Imagine you want to wrap a Christmas present. The box is 3D. The wrapping paper is 2D.

Texture Mapping is exactly this process in reverse. We don't paint on the 3D object; we take the 3D object, slice it open, and flatten it onto a 2D image like a piece of origami paper.



The Shift

We are not just builders; we are tailors. We must learn to stitch 2D images onto 3D skeletons.

So far, our objects have simple, uniform colours. How do we add rich surface detail like wood grain, brick patterns, or character faces?

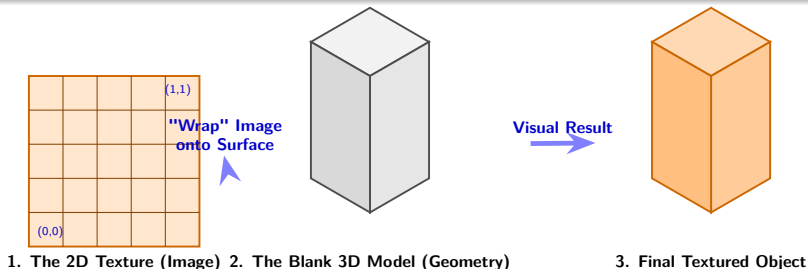
- Creating more triangles to model these details would be computationally expensive and artistically difficult.

So far, our objects have simple, uniform colours. How do we add rich surface detail like wood grain, brick patterns, or character faces?

- Creating more triangles to model these details would be computationally expensive and artistically difficult.
- **Solution:** Texture Mapping!

Definition: Texture Mapping

A technique that "wraps" a 2D image, called a **texture**, onto the surface of a 3D model. This allows us to add immense visual detail with very simple underlying geometry.



To control how a 2D image wraps around a 3D shape, we need a coordinate system.

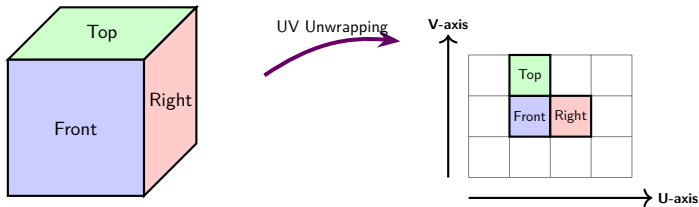
- Each vertex of the 3D mesh is assigned a 2D coordinate, called a **UV coordinate**.
- The **U** coordinate corresponds to the horizontal (X) axis of the texture.
- The **V** coordinate corresponds to the vertical (Y) axis of the texture.
- These UVs are stored as another vertex attribute.

UV Unwrapping

The process of creating this mapping is called **UV unwrapping**. It's like carefully cutting the seams of a 3D object and laying it flat, a task typically done by 3D artists.

Visualising UV Unwrapping

A 3D cube is "unwrapped" into a 2D UV layout. Each vertex on the cube has a corresponding (u, v) coordinate on this 2D map.



During rendering, the GPU smoothly **interpolates** these UVs across the surface of each triangle, giving a unique texture coordinate to every single pixel.

Vertex Shader

- Receives the per-vertex UV coordinate as an attribute ('a_texCoord').
- Its main job is to simply pass this coordinate on to the fragment shader via a 'varying'.

```
// Vertex Shader
attribute vec2 a_texCoord;
varying vec2 v_texCoord;

void main() {
    //... usual position calculation ...

    // Pass UV to fragment shader
    v_texCoord = a_texCoord;
}
```

Fragment Shader

- Receives the interpolated 'v_texCoord' for the current pixel.
- Accesses the texture via a special 'uniform sampler2D'.
- Uses the 'texture2D()' function to look up the colour from the texture at that coordinate. This is called **sampling**.

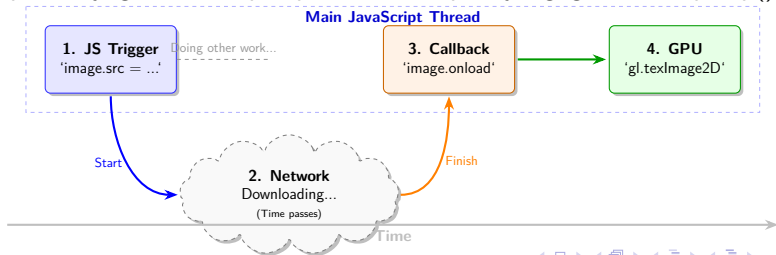
```
// Fragment Shader
varying vec2 v_texCoord;
uniform sampler2D u_texture;

void main() {
    vec4 textureColor =
        texture2D(u_texture, v_texCoord);
    gl_FragColor = textureColor;
}
```

Loading an image from a server is an **asynchronous** operation. We must wait for it to download before uploading it to the GPU.

The process:

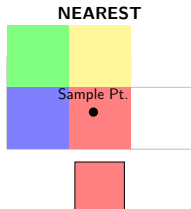
- 1 Create a WebGL texture object: `gl.createTexture()`.
- 2 Create an HTML `'Image'` object and set its `'src'` to start the download.
- 3 **Crucially**, use the `'image.onload'` event handler. All GPU texture operations happen inside this callback function.
- 4 Inside `'onload'`, bind the texture and upload the image data to the GPU using `'gl.texImage2D()'`.
- 5 Optionally, generate mipmaps for better quality: `'gl.generateMipmap()'`.



What happens when a UV coordinate falls *between* the discrete pixels (texels) of a texture? Filtering rules tell the GPU how to decide the colour.

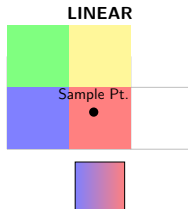
gl.NEAREST

- **Method:** Grabs the colour of the single nearest texel.
- **Result:** Fast, but creates a blocky, pixelated look when the texture is magnified.
- **Use Case:** Retro-style graphics.



gl.LINEAR

- **Method:** Takes the four nearest texels and returns a weighted average of their colours. (Bilinear Filtering)
- **Result:** Much smoother and less blocky.
- **Use Case:** Most general-purpose rendering.



Wrapping rules determine what happens when a UV coordinate falls outside the standard $[0, 1]$ range.

`gl.REPEAT`



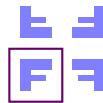
The texture tiles or repeats itself. Perfect for patterns.

`gl.CLAMP_TO_EDGE`



The edge colour of the texture is stretched outwards.

`gl.MIRRORED_REPEAT`



The texture repeats, but every other repetition is mirrored.

These parameters are set once per texture object using `'gl.texParameteri()'`.

Texture Modulation

Integrating Surface Detail with 3D Shape

A texture on its own is just a flat image. To make it look like a 3D surface, it must react to light.

The Logic: Modulation

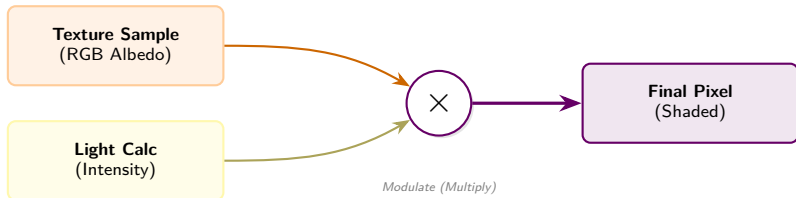
We treat the texture as the object's **Albedo** (base paint). We then **modulate** (multiply) this color by the light intensity calculated by our shading model.

Fragment Shader Steps

- 1 **Sample** the texture at current UV.
- 2 **Calculate** light intensity (N·L).
- 3 **Multiply** texture RGB by Light.
- 4 **Output** the shaded pixel.

The Equation:

$$C_{final} = C_{texture} \times (I_{ambient} + I_{diffuse} + I_{specular})$$



Shader: Texturing + Lighting

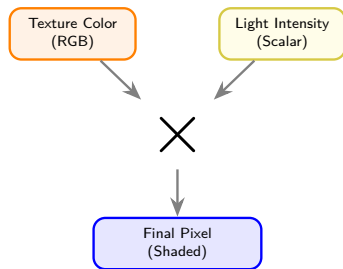
The "Multiplication" Logic

In the fragment shader, texturing is simply retrieving a color and **modulating** (multiplying) it by the light intensity.

```

1 // --- Fragment Shader ---
2 precision mediump float;
3 // Varyings from Vertex Shader
4 varying vec2 v_texCoord;
5 varying vec3 v_normal;
6
7 // Uniforms
8 uniform sampler2D u_texture;
9 uniform vec3 u_lightColor;
10
11 void main() {
12     // 1. Sample texture (The "Paint")
13     vec4 albedo = texture2D(u_texture, v_texCoord);
14
15     // 2. Calc Light (The "Brightness")
16     vec3 norm = normalize(v_normal);
17     // Dot product = angle of light
18     float diff = max(dot(norm, u_lightDirection), 0.0);
19     vec3 diffuse = diff * u_lightColor;
20
21     // 3. COMBINE: Paint * Brightness
22     vec3 finalColor = diffuse * albedo.rgb;
23     gl_FragColor = vec4(finalColor, albedo.a);
24 }

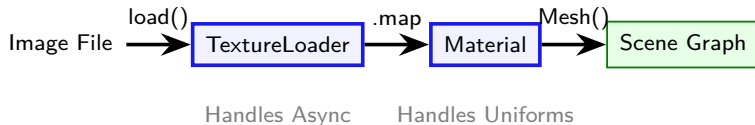
```



Abstraction: Texture Mapping in Three.js

Three.js automates the complex WebGL steps (creating texture buffers, binding uniforms, handling async image loading) into a declarative API.

```
1 // 1. Create a Loader (Handles AJAX/Async logic internally)
2 const textureLoader = new THREE.TextureLoader();
3
4 // 2. Load the texture (Returns a Texture object)
5 const colorTexture = textureLoader.load('path/to/my_texture.png');
6
7 // 3. Create a Material (Smart Shader Wrapper)
8 // 'map' automatically binds the texture to the right uniform
9 const material = new THREE.MeshStandardMaterial({
10   map: colorTexture, // Base Color
11   roughness: 0.5    // PBR Property
12 });
13
14 // 4. Mesh = Geometry + Material
15 const texturedMesh = new THREE.Mesh(geometry, material);
16 scene.add(texturedMesh);
17
```



Optimisation: Mip-Mapping

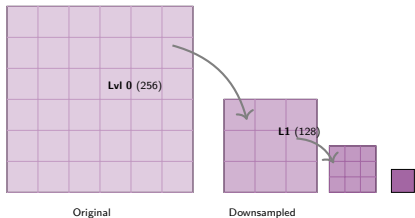
Solving Minification Aliasing & Memory Costs

The Problem: Minification

When an object is far away, one screen pixel covers hundreds of texture pixels. Sampling just one causes visual noise (**Aliasing**).

The Solution: Mip-Maps

Pre-calculate a chain of lower-resolution images (levels). The GPU selects the best fit based on distance.



Memory Cost Analysis

Each level is 1/4 the area of the previous.

$$\text{Total} = 1 + \frac{1}{4} + \frac{1}{16} + \dots \approx 1.33$$

Result: Only 33% extra VRAM usage.

Why Power of 2?

Mip-mapping generally requires texture dimensions to be powers of two (2^n , e.g., 256, 512) so they can be divided cleanly down to 1×1 .

Implementation (WebGL):

```
1 // 1. Upload Base Level 0
2 gl.texImage2D(..., image);
3 // 2. Generate the chain (Hardware accelerated)
4 gl.generateMipmap(gl.TEXTURE_2D);
5 // 3. Set Filter to use them
6 gl.texParameteri(gl.TEXTURE_2D,
7   gl.TEXTURE_MIN_FILTER,
8   gl.LINEAR_MIPMAP_LINEAR);
9
```

The Aliasing Problem

The Artifact: When a distant pixel covers many texels, sampling just one causes "shimmering" or Moiré patterns (Undersampling).

The Fix: Mip-maps provide pre-averaged colors, ensuring the pixel represents the *average* of the underlying details.

The Aliasing Problem

Why do diagonal lines look like stairs?

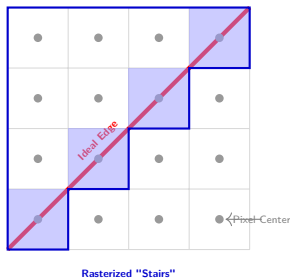
What is Aliasing?

In rasterisation, we are trying to represent continuous, infinite-resolution geometry using a discrete, finite grid of pixels.

The Cause: Undersampling

The center of a pixel is either **inside** or **outside** a triangle.

- This binary decision (0 or 1) creates hard, jagged transitions known as "jaggies".
- High-frequency details (thin lines, sharp edges) are lost because the sampling rate (screen resolution) is too low.



The "Staircase" Effect

A smooth diagonal line becomes a sequence of blocks because pixels cannot display partial coverage natively.

Resolving Aliasing: Anti-Aliasing (AA)

Techniques to Smooth the Edges

1. Supersampling (SSAA)

The Brute Force Method

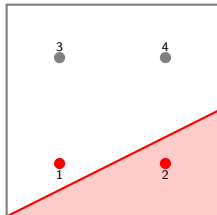
- Render the scene at **4x resolution** internally.
- Downscale (average) it to the screen size.
- **Pros:** Highest quality.
- **Cons:** Extremely slow (4x Fragment Shader cost).

2. Multisample AA (MSAA)

The Hardware Standard

- Only supersamples the **depth/stencil** buffers at edges.
- Fragment Shader runs **once** per pixel (center), but color is stored in multiple "sub-samples" if edges cover the pixel.
- **Result:** Smooth edges, standard cost for textures.

Single Pixel (4x MSAA)



Logic:

- Triangle covers 2 of 4 samples (50%).
- Shader runs once (Red).
- Final Pixel Color = 50% Red, 50% Background.

3. Post-Process (FXAA)

Blur jagged edges *after* the frame is rendered. Fast but blurry.

Procedural Textures: HTML5 Canvas

Generating Assets on the CPU

What is it?

Instead of loading a static image (PNG), we can use the browser's 2D drawing API to paint text, shapes, or graphs into memory, then upload that as a texture.

Use Cases:

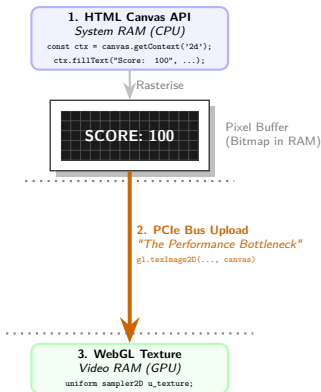
- Dynamic Text Labels (Billboards).
- In-game Scoreboards / Dashboards.
- Procedural noise or gradients.

Code Implementation:

```

1 // 1. Create Canvas & Draw (CPU)
2 const canvas = document.createElement('canvas');
3 const ctx = canvas.getContext('2d');
4 ctx.fillStyle = "red";
5 ctx.fillRect(0, 0, 512, 512);
6 ctx.fillStyle = "white";
7 ctx.fillText("Score: 100", 50, 50);
8
9 // 2. Create Texture from Canvas (GPU)
10 const texture = gl.createTexture();
11 gl.bindTexture(gl.TEXTURE_2D, texture);
12 // WebGL accepts the canvas element directly!
13 gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA,
14              gl.RGBA, gl.UNSIGNED_BYTE, canvas);

```



Performance Warning

Uploading (Step 2) is slow. Do not re-generate the canvas texture every frame unless necessary.

Texture Mapping Application: Billboarding

Cheating 3D Complexity with 2D Sprites

1. The Concept

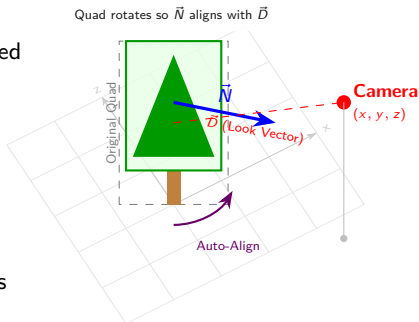
- A **Billboard** is a 2D quad that automatically rotates to face the camera.
- **Use Cases:** Particles (smoke, fire), detailed foliage (trees), or UI labels.

2. The Math

- We calculate the **Look Vector** $\vec{D}$:

$$\vec{D} = \text{normalize}(P_{cam} - P_{obj})$$

- We construct a Model Matrix that rotates the object's normal $\vec{N}$ to align with $\vec{D}$.



Simulating Surface Detail: Bump Mapping

What is it?

Adding micro-detail (bricks, wrinkles) without adding polygons. It fakes depth by "tricking" the lighting engine.

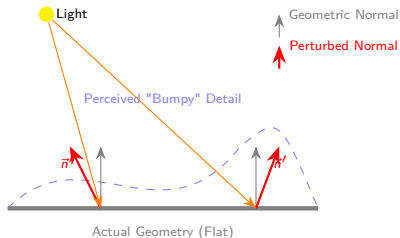
How it works:

We use a grayscale **Height Map**.

- **Values:** Bright = High, Dark = Low.
- **Effect:** The shader calculates the gradient (slope) to tilt the **Surface Normal**.

Implementation (Simplified Concept):

```
1 // 1. Sample height at current and neighbor pixels
2 float h = texture(u_heightMap, uv).r;
3 float hRight = texture(u_heightMap, uv + vec2(eps, 0)).r;
4 float hUp = texture(u_heightMap, uv + vec2(0, eps)).r;
5
6 // 2. Compute "tilt" (Gradient)
7 vec3 bump = vec3(h - hRight, h - hUp, 1.0);
8
9 // 3. Perturb the original normal
10 vec3 normal = normalize(v_normal + bump);
11
```



Faking Geometry: Normal Mapping

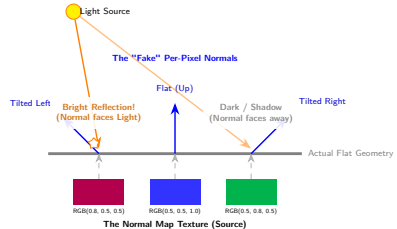
What is it?

A technique to simulate high-resolution details (bumps, dents) on a flat, low-poly surface without adding triangles.

How it works (The "Fake"):

We encode 3D surface angles into a 2D color texture.

- The fragment shader looks up color values to bend the normal vector *per-pixel*.
- **The Mapping:** RGB [0, 1] remapped to XYZ [-1, 1].
- Flat surface = Solid Blue
(0.5, 0.5, 1.0) $\rightarrow$ (0, 0, 1).



Implementation Fragment Shader:

```

1 // 1. Get RGB color from texture (range 0.0 to 1.0)
2 vec3 mapN = texture(u_normalMap, v_uv).rgb;
3
4 // 2. Remap from [0,1] range to [-1,1] for vector math
5 // (e.g., 0.5 becomes 0.0, 0.0 becomes -1.0)
6 mapN = mapN * 2.0 - 1.0;
7
8 // 3. Use this NEW, bent normal for lighting
9 // instead of the actual geometric normal (v_normal)
10 float diff = max(dot(mapN, lightDir), 0.0);
11

```

From Grayscale Pixels to 3D Vectors

Normal Map Image Generation

1. Gradient Approximation (Sobel-like Kernel) To find the surface "tilt," we sample a 3×3 pixel neighborhood around each pixel (x, y) to compute horizontal and vertical slopes (dx, dy) .

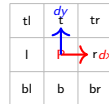
$$dx = (tr + 2r + br) - (tl + 2l + bl)$$

$$dy = (bl + 2b + br) - (tl + 2t + tr)$$

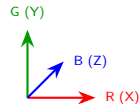
$$dz = 1.0/\text{strength}$$

2. Normalisation & RGB Mapping The raw gradient (dx, dy, dz) is turned into a unit vector. Since RGB values range from $[0, 1]$ and vectors from $[-1, 1]$, we remap the data:

Neighborhood Sampling



Vector Result (RGB)



Standard Tangent Space:
Typical "Purple" Tint

```
1 len = sqrt(dx*dx + dy*dy + dz*dz);
2 R = ((dx/len) + 1.0) * 127.5; // X-axis
3 G = ((dy/len) + 1.0) * 127.5; // Y-axis
4 B = ((dz/len) + 1.0) * 127.5; // Z-axis (Up)
```

- **Strength Factor:** A lower strength value in the code increases dz , making the normal map appear "flatter".
- **Final Look:** The Z-axis (pointing "out" from the surface) is mapped to Blue, which is why neutral normal maps are primarily light blue $(0.5, 0.5, 1.0)$.

What is it?

Unlike Normal Mapping (which is an optical illusion), this technique **physically moves vertices** to change the silhouette of the object.

How it works:

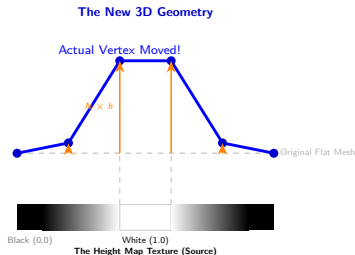
We read a texture in the **Vertex Shader**.

- **Black Pixels:** Vertex stays at 0.
- **White Pixels:** Vertex pushed fully outward along Normal.

Note: Requires a high-density mesh!

Implementation (Vertex Shader):

```
1 uniform sampler2D u_heightMap;
2
3 void main() {
4     // 1. Read height (0.0 to 1.0)
5     float h = texture(u_heightMap, uv).r;
6
7     // 2. Move position along Normal
8     // "scale" determines max height
9     vec3 newPos = position + (normal * h * scale);
10
11     gl_Position = projection * view * vec4(newPos, 1.0);
12 }
```



Reflections: Environment Mapping

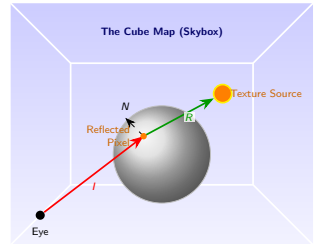
What is it?

Simulating mirror-like reflections of the surrounding world (Skybox) without ray-tracing the whole scene.

How it works:

We use a **Cube Map**: 6 textures forming a box around the scene.

- ➊ Calculate the Reflection Vector (R) based on the camera view.
- ➋ Use R as a 3D coordinate to "hit" a pixel on the enclosing cube.
- ➌ Paint that pixel's color onto the object.



Implementation:

```

1 // u_skybox is a collection of 6 textures
2 uniform samplerCube u_skybox;
3
4 void main() {
5     // 1. Vector from Eye to Surface
6     vec3 I = normalize(v_pos - u_camPos);
7
8     // 2. Calculate Reflection Vector
9     vec3 R = reflect(I, normalize(v_normal));
10
11     // 3. Sample the Cube Map using 3D vec R
12     // The GPU automatically finds the right face
13     gl_FragColor = texture(u_skybox, R);
14 }
15

```

Comparing Texture Mapping Techniques

Illusion vs. Reality in Modern Rendering

Mapping Technique	Definition	Geometry	Characteristics	Illustration
Bump	Grayscale height map perturbs normals for lighting.	No Change	Oldest; "bumpy" look but edges remain flat.	
Normal	RGB textures "bend" normals per-pixel (XYZ).	No Change	Industry standard for high-poly faking.	
Displacement	Vertices physically moved along normal via height.	Altered	Changes silhouette; needs high-density mesh.	
Environment	Projects Cube Map/Skybox onto surface.	No Change	Mirror-like reflections via reflection vectors.	

The Optical Illusion (Bump/Normal)

These techniques rely on the dot product of the "faked" normal and light direction to calculate color.

$$I = \max(\text{dot}(\mathbf{n}_{\text{map}}, \mathbf{l}), 0.0)$$

The Geometric Reality (Displacement)

Vertex positions are updated in the Vertex Shader before rasterization.

$$\mathbf{P}_{\text{new}} = \mathbf{P}_{\text{orig}} + (\mathbf{n} \times h \times \text{scale})$$

Note: Normal maps remap RGB [0, 1] to XYZ [-1, 1] for per-pixel lighting calculations.

Bump Mapping vs. Normal Mapping

Shader Implementation

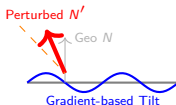
Bump Mapping (Height Map)

```

1 float h = texture(m, uv).r;
2 float hx = texture(m, uv+e.x).r - h;
3 float hy = texture(m, uv+e.y).r - h;
4 vec3 n = normalize(vN + vec3(hx, hy, 0));
5

```

Logic: Implemented in the **Fragment Shader**, it samples grayscale values to find a height gradient (∇h); the resulting "tilt" perturbs the surface normal per-pixel to fake depth.



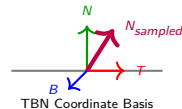
Normal Mapping (Vector Map)

```

1 vec3 rgb = texture(nMap, uv).rgb * 2. - 1.;
2 mat3 TBN = mat3(vT, vB, vN);
3 vec3 n = normalize(TBN * rgb);
4

```

Logic: Implemented in the **Fragment Shader**, it replaces the geometric normal with a high-resolution vector from an RGB texture, using a **TBN Matrix** to transform the vector from Tangent Space to World Space.



- **Note:** Geometry is **not** altered; edges remain flat.
- **Context:** Ideal for micro-details on moving **Objects**.

Displacement vs. Environment Mapping

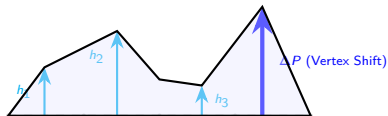
Shader Implementation

Displacement Mapping

```
1 float h = texture(dMap, uv).r;
2 // Move vertex along normal
3 vec3 newP = pos + (normal * h * scale);
4 gl_Position = proj * mv * vec4(newP, 1.0);
5
```

Logic: Implemented in the **Vertex Shader**, it uses a height map to physically offset vertex positions ($P' = P + \mathbf{n} \cdot h$), which effectively alters the mesh silhouette and geometry.

$$P' = P + (\mathbf{n} \cdot h)$$

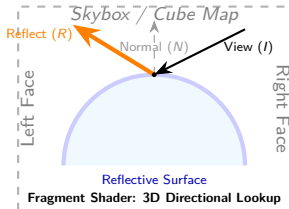


Original Mesh (P)

Environment Mapping (Reflections)

```
1 vec3 I = normalize(vWPos - camera);
2 vec3 R = reflect(I, normalize(vN));
3 // Sample 3D Cube Map direction
4 vec4 col = textureCube(eMap, R);
5
```

Logic: Implemented in the **Fragment Shader**, it calculates a reflection vector based on the camera position and surface normal to perform a 3D lookup within a **Cube Map** (Skybox).



- **Note:** Displacement requires high vertex density to prevent "tearing".

Question:

How do we accelerate hundreds of point lights?

The Scenario: Imagine expanding our hilly village. We want hundreds of lampposts, each casting realistic light and shadows across the terrain and houses.

The Problem:

- Point lights are computationally expensive because their illumination must be evaluated for every fragment in all directions.
- In WebGL and Three.js, adding too many active lights simultaneously exceeds the hardware's maximum uniform vector limits, which can crash the shader entirely.
- When shadows are enabled, a single point light must render the scene from 6 different perspectives (a shadow cube map) to calculate occlusions.

The Challenge: How can we maintain the visual fidelity of a fully illuminated night-time village without bringing the framerate to a crawl or crashing the WebGL context?

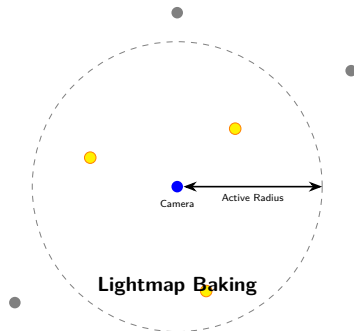
1. Distance-Based Culling (Active Limits)

- We calculate the squared distance from every lamppost to the camera.
- We sort the lights and only enable the closest ones (e.g., a maximum of 10 active lamps at a time).
- Distant lamps are disabled, saving massive computational resources.

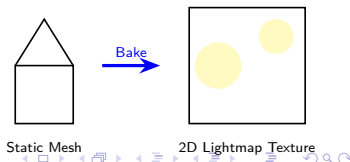
2. Light Baking (Lightmaps)

- For static objects (houses, trees, terrain), we pre-calculate the illumination and save it into a 2D texture.
- The shader simply samples this texture (using a second set of UV coordinates) instead of running complex light equations.
- Real-time lights are then turned off entirely for static geometry.

Distance Culling

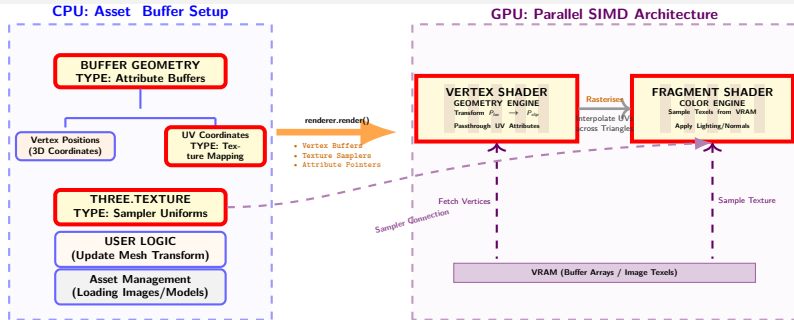


Lightmap Baking



Meshing & Texture Mapping (Three.js)

Identifying Geometry Components & Mapping Responsibilities



Mesh & Texture Mapping Responsibilities

- **Buffer Geometry:** The CPU stores raw shape data (positions) and mapping data (UVs) in high-performance `Float32Arrays`.
- **UV Mapping:** A 2D coordinate system (u, v) defined per-vertex that tells the GPU which part of an image belongs on which part of the mesh.
- **Vertex Shader:** Responsible for receiving vertex attributes and passing the UV coordinates to the next stage.
- **Fragment Shader:** Performs the final "Magic Trick" it uses the interpolated UVs to look up (sample) colors from the texture stored in VRAM.

- **Meshes** are the foundation of 3D models, defined by **vertices** (points with attributes) and **indices** (connectivity).
- Using an **Index Buffer (IBO)** is crucial for memory efficiency by allowing vertices to be reused.
- We transfer geometry from the CPU to the GPU's **VBO** and **IBO** and use **attribute pointers** to describe the data layout to the shaders.
- **Texture Mapping** adds surface detail by wrapping a 2D image onto a 3D model using **UV coordinates**.
- **Filtering** and **Wrapping** parameters control how textures are sampled and behave.
- For realistic results, texture colour (**albedo**) is modulated by a lighting model.

The Grand Finale: Geometry Meets Texture

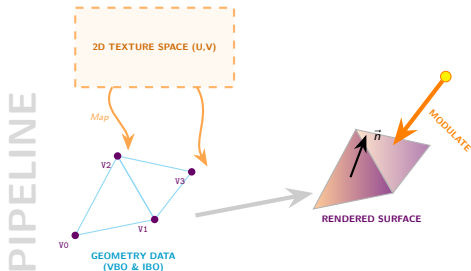
The Professional's Perspective

The Digital Craft

- **The Skeleton (Geometry):** We build the **Wireframe Ghost** using VBOs and IBOs. It is pure mathefficient, rigid, and hollow.
- **The Skin (Texture):** We wrap the ghost in **Digital Origami**. Through UV mapping, we decouple visual detail from geometric complexity.
- **The Breath (Shaders):** Lighting **modulates** the surface, turning flat pixels into a believable reality.

The Professional Insight

Great graphics are not just about high polygon counts. They are about the **efficiency** of the data and the **intelligence** of the mapping.



THE GRAPHICS SYNTHESIS

"Geometry provides the structure; Textures provide the soul."

Master the Pipeline. Build the World.

Questions?

Study Guide: Mastering the Pipeline

A Roadmap from Points to Pixels

To master this lecture, follow the **Data Flow**. Understand how a point on your CPU becomes a pixel on your screen.

Phase 1: Geometric Structure

- Focus on the **Triangle** as the fundamental primitive.
- Understand **Index Buffers (IBO)** as a memory-saving "recipe".

Phase 2: The GPU Handshake

- Master **Stride** and **Offset** to describe raw memory layout.
- Visualise data moving from JS arrays to **VRAM**.

Phase 3: Surface & Textures

- Treat **UV Mapping** as a 2D-to-3D translation.
- Learn how lighting **modulates** (multiplies) texture colors.

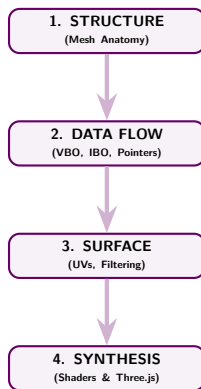


Figure: The 4-step learning path for this lecture.

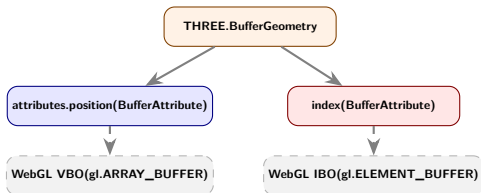
Appendix A: VBOs & IBOs in Three.js

Mapping Theory to Practice

Three.js wraps raw WebGL calls into a class called `BufferGeometry`. Understanding VBOs allows you to build custom procedural shapes (like terrain or clouds) efficiently.

The Translation Dictionary

- **VBO** → `THREE.BufferAttribute`
- **IBO** → `geometry.index`
- **VAO** → `THREE.BufferGeometry`



Why use BufferGeometry?

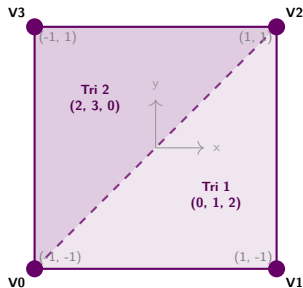
- **Performance:** It stores data in `Float32Array` (Typed Arrays), which maps directly to GPU memory.
- **Control:** You define every single vertex and triangle manually.

Appendix B: Custom Mesh Implementation

Creating a "Quad" from Scratch

To create custom geometry, we define the **Vertices (VBO)** and the **Indices (IBO)** manually using Typed Arrays.

```
1 // 1. Define Vertices (x, y, z) - THE VBO DATA
2 // A simple square (4 vertices)
3 const vertices = new Float32Array([
4     -1.0, -1.0, 0.0, // V0: Bottom-Left
5     1.0, -1.0, 0.0, // V1: Bottom-Right
6     1.0, 1.0, 0.0, // V2: Top-Right
7     -1.0, 1.0, 0.0 // V3: Top-Left
8 ]);
9
10 // 2. Define Indices - THE IBO DATA
11 // Two triangles sharing V0 and V2
12 const indices = [
13     0, 1, 2, // Triangle 1
14     2, 3, 0 // Triangle 2
15 ];
16
17 // 3. Create the Geometry Container
18 const geometry = new THREE.BufferGeometry();
19
20 // 4. Attach VBO (ItemSize = 3 for x,y,z)
21 geometry.setAttribute( 'position',
22     new THREE.BufferAttribute( vertices, 3 )
23 );
24
25 // 5. Attach IBO
26 geometry.setIndex( indices );
27
28 // 6. Create Mesh
29 const mesh = new THREE.Mesh( geometry, material );
30
```



Key Takeaway

By using an Index Buffer (lines 11-14), we defined a square with only 4 vertices instead of 6, saving 33% memory.

Appendix C: Why Triangles?

The Fundamental Primitive of Computer Graphics

In computer graphics, why is every 3D model, no matter how complex, ultimately broken down into triangles?

1. Guaranteed Planarity

- **Mathematical Fact:** Any three non-collinear points define a single plane.
- **Benefit:** A triangle is *a/ways* flat. It cannot be bent or warped.
- **Contrast:** A Quad (4 points) can be "non-planar" (bent), making rendering unpredictable.

2. Well-Defined Normals

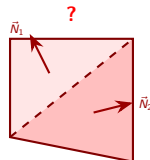
- Because it is flat, a triangle has a single, constant **Normal Vector** across its surface.
- **Benefit:** This simplifies lighting calculations (dot products) significantly.

3. GPU Optimisation

- **Rasterisation:** GPU hardware is hard-wired to process triangles.
- **Barycentric Coordinates:** Interpolating attributes (colour, UVs) across a triangle is computationally efficient and mathematically robust.
- **Convexity:** Triangles are always convex, ensuring simple fill algorithms.



Triangle
(Always Flat)



Bent Quad
(Ambiguous Normal)

Appendix D: Anti-Aliasing in Three.js

Hardware (MSAA) vs. Post-Processing (FXAA)

1. Hardware MSAA (The Easy Way)

The simplest method is to request the browser's default WebGL Anti-Aliasing (usually 4x MSAA) when creating the renderer.

```

1 // 1. Enable 'antialias' in the constructor
2 const renderer = new THREE.WebGLRenderer({
3   antialias: true, // Requests MSAA
4   alpha: true
5 });
6
7 // 2. Adjust Pixel Ratio for High-DPI screens
8 renderer.setPixelRatio(window.devicePixelRatio)
9   ;
10
11 // 3. Render Loop
12 function animate() {
13   // Standard render call
14   renderer.render(scene, camera);
15 }
16

```

Limitations

MSAA is not available if you use **Deferred Rendering** or complex **Post-Processing** chains, as they render to off-screen textures (RenderTargets) which often don't support MSAA in WebGL 1/2.

2. Post-Processing FXAA

If you use an EffectComposer (for bloom, depth of field, etc.), you lose built-in MSAA. You must add an FXAA pass to smooth edges cheaply at the end.

```

1 import { EffectComposer } from 'three/.../EffectComposer.
   js';
2 import { RenderPass } from 'three/.../RenderPass.js';
3 import { ShaderPass } from 'three/.../ShaderPass.js';
4 import { FXAAShader } from 'three/.../FXAAShader.js';
5
6 // 1. Setup Composer
7 const composer = new EffectComposer(renderer);
8
9 // 2. Add Base Scene Render
10 const renderPass = new RenderPass(scene, camera);
11 composer.addPass(renderPass);
12
13 // 3. Add FXAA Shader Pass
14 const fxaaPass = new ShaderPass(FXAAShader);
15 // *Crucial*: Set resolution uniform!
16 const pixelRatio = renderer.getPixelRatio();
17 fxaaPass.material.uniforms['resolution'].value.x =
18   1 / (window.innerWidth * pixelRatio);
19 fxaaPass.material.uniforms['resolution'].value.y =
20   1 / (window.innerHeight * pixelRatio);
21 composer.addPass(fxaaPass);
22
23 // 4. Render Loop
24 function animate() {
25   composer.render(); // Use composer, NOT renderer
26 }
27

```